

SYNCHRONIZATION :

22 February 2025 16:32

Synchronization Introduction

1. What is Synchronization ?

It is the process by which we control the accessibility of multiple threads to a particular shared resource

2. Problem which can occur without synchronization :-

1. Data Inconsistency
2. Thread interference

3. Advantages of Synchronization :-

1. No data inconsistency problem
2. No thread interference

4. Disadvantages of Synchronization :-

1. Increases the waiting time period of threads
2. Create performance problems

1. What is Synchronization?

Synchronization in Java is a mechanism that allows only one thread to access a shared resource at a time. It helps prevent **data inconsistency** and **race conditions** when multiple threads try to modify shared data.

2. Need for Synchronization

- **Prevent Data Corruption:** Ensures correct values are updated when multiple threads access shared data.
- **Avoid Race Conditions:** Prevents situations where multiple threads modify data simultaneously, leading to unexpected results.
- **Ensure Consistency:** Makes sure that the operations on shared resources happen in a controlled manner.

3. Real-Life Example of Synchronization

Example: ATM Withdrawal

Imagine an **ATM machine** where two people (threads) try to withdraw money from the same bank account (shared resource).

Without Synchronization (Problem)

- Two people check the balance at the same time.
- Both see ₹10,000 in the account.
- Both withdraw ₹8,000 simultaneously.
- The final balance should be ₹2,000, but since both checked before withdrawing, the system may allow both to withdraw ₹8,000, leading to an incorrect balance of ₹-6,000.

With Synchronization (Solution)

- One person (thread) locks the account.
- Withdraws money and updates the balance.
- Unlocks the account.
- The next person (thread) can now access it safely.

EXAMPLE :

```
package multithreading;

class BankAccount {
    int balance = 10000;

    void withdraw(int amount) {
        if (balance >= amount) {
            System.out.println(Thread.currentThread().getName() + " is withdrawing " + amount);
            balance -= amount;
            System.out.println("Remaining balance: " + balance);
        } else {
```

```

        System.out.println(Thread.currentThread().getName() + " cannot withdraw. Insufficient balance!");
        System.out.println("Current balance: " + balance);
    }
}

public class BankApp extends Thread {

    static BankAccount bankAccount;
    int amount;

    @Override
    public void run() {
        bankAccount.withdraw(amount);
    }

    public static void main(String[] args) {
        bankAccount = new BankAccount();

        BankApp jatin = new BankApp();
        jatin.amount = 8000;
        jatin.setName("Jatin");
        jatin.start();

        BankApp anjali = new BankApp();
        anjali.amount = 8000;
        anjali.setName("Anjali");
        anjali.start();
    }
}

```

4. How to Achieve Synchronization in Java?

Java provides three ways to achieve synchronization:

1. Synchronized Method
2. Synchronized Block

1. Synchronized Method

- The entire method is **locked** when one thread executes it.
- Other threads **must wait** until the first thread finishes execution.
- Locks **on the object level** (for instance methods) or **class level** (for static methods).

🚫 **Drawback:** It locks the entire method, even if only part of it needs protection.

- **Internal Working of Synchronized Method in Java**

```

package multithreading;

class BankAccount {
    int balance = 10000;

    synchronized void withdraw(int amount) {
        if (balance >= amount) {
            System.out.println(Thread.currentThread().getName() + " is withdrawing " + amount);
            balance -= amount;
            System.out.println("Remaining balance: " + balance);
        } else {
            System.out.println(Thread.currentThread().getName() + " cannot withdraw. Insufficient balance!");
            System.out.println("Current balance: " + balance);
        }
    }
}

public class BankApp extends Thread {

    int amount;
    static BankAccount bankAccount;

    @Override
    public void run() {
        bankAccount.withdraw(amount);
    }
}

```

```

    }

    public static void main(String[] args) {
        BankAccount bankAccount = new BankAccount();

        BankApp jatin = new BankApp();
        jatin.amount = 8000;
        jatin.setName("Jatin");
        jatin.start();

        BankApp anjali = new BankApp();
        anjali.amount = 8000;
        anjali.setName("Anjali");
        anjali.start();
    }
}

```

When you use `synchronized` on a method in Java, it ensures **mutual exclusion**, meaning only one thread can access that method at a time for a particular object.

Step-by-Step Working of Your Code

Step 1: Initial Setup

- **Main thread starts execution** and initializes a **single BankAccount object** (shared by both threads).
- Two threads (Jatin and Anjali) are created with different withdrawal amounts.
- Both threads call `start()`, which invokes the `run()` method.

Step 2: Entering the `withdraw()` Method

- Both threads (Jatin and Anjali) **try to call `withdraw(int amount)`** at the same time.
- Since the method is **synchronized**, **only one thread can enter** at a time.

Thread Execution Order (Depends on CPU Scheduling)

1. **If Jatin gets the CPU first**, it enters the `withdraw()` method and locks the `BankAccount` object.
 - **Other threads must wait** until Jatin releases the lock.
 - Jatin checks if balance is sufficient.
 - If yes, it deducts the amount and prints the new balance.
 - **Once completed, Jatin releases the lock.**
2. **Now Anjali gets the CPU**, enters `withdraw()`, and locks the `BankAccount` object.
 - **If balance is insufficient now (because Jatin already withdrew)**, Anjali sees an error message.
 - **Anjali releases the lock** after execution.

Step 3: Locking Mechanism

- The `synchronized` keyword makes sure that **only one thread at a time can execute `withdraw()`**.
- Java uses **intrinsic locks (or monitors)** internally for synchronization:
 - **When a thread enters a synchronized method**, it **acquires a lock on the object** (`bankAccount` in this case).
 - **Other threads trying to enter the method must wait** until the lock is released.
 - **After execution, the lock is released**, allowing the next thread to proceed.

2. Synchronized Block

- Only a **specific part of the method** is synchronized instead of the entire method.
- More **efficient** than synchronized methods.
- Useful when only **critical code needs synchronization** while allowing other operations to run freely.

EXAMPLE:

```

package multithreading;
class BankAccount2 {
    private int balance = 10000;

    void withdraw(int amount) {
        System.out.println(Thread.currentThread().getName() + " is checking balance...");

        synchronized (this) { // Synchronizing only the critical section
            if (balance >= amount) {
                System.out.println(Thread.currentThread().getName() + " is withdrawing " + amount);
                try { Thread.sleep(1000); } catch (InterruptedException e) {}
                balance -= amount;
                System.out.println("Remaining balance: " + balance);
            } else {
                System.out.println(Thread.currentThread().getName() + " cannot withdraw. Insufficient
balance!");
            }
        }
    }
}

```

```

    }
}

public class SynchronizedBlockDemo extends Thread {
    static BankAccount2 account;
    int amount;

    @Override
    public void run() {
        account.withdraw(amount);
    }

    public static void main(String[] args) {
        account = new BankAccount2();

        SynchronizedBlockDemo t1 = new SynchronizedBlockDemo();
        t1.amount = 8000;
        t1.setName("Jatin");
        t1.start();

        SynchronizedBlockDemo t2 = new SynchronizedBlockDemo();
        t2.amount = 8000;
        t2.setName("Anjali");
        t2.start();
    }
}

```

💡 **Advantage:** Synchronizes **only the required block** instead of locking the entire method.

Feature	Synchronized Method	Synchronized Block
Locks Entire Method?	✅ Yes	❌ No
Efficiency	🔴 Less Efficient (Locks full method)	🟢 More Efficient (Locks only critical section)
Execution Speed	🟡 Slower	⚡ Faster
Best Use Case	When the entire method must be protected	When only part of the method needs synchronization

3. Static synchronization:

What is Static Synchronization?

- Normal synchronized methods work on **object level locks** — meaning they lock **one object at a time**.
- static synchronized works on **class level locks** — meaning they lock **the whole class**, not just one object.

Real-Life Example of Static Synchronization & Multiple Objects

Example: Online Railway Ticket Booking System

💡 Scenario:

Imagine multiple railway booking counters at different locations (Delhi, Mumbai, Bangalore, etc.), all connected to a **central server**. The **total number of seats** for a train is **stored centrally** (shared resource).

🔑 Key Points:

- Multiple counters (objects) are handling seat booking.
- The **seat count must be updated globally** across all counters.
- **Race condition** can occur if multiple people try booking at the same time.
- **Static Synchronization is required** to ensure only one booking request is processed at a time.

CODE:

```

package multithreading;

class BookTheaterSeat {

```

```

static int totalSeats = 20;

synchronized static void bookSeats(int seats) {

    if (totalSeats >= seats) {

        System.out.println(seats + " Seats Book successfully");
        totalSeats -= seats;
        System.out.println(totalSeats + " No of seats left");
    } else {
        System.out.println(seats + " Seats cannot be booked");
        System.out.println("No of seats left " + totalSeats);
    }

}

}

class Thread1 extends Thread {

    BookTheaterSeat b;
    int seats;

    public Thread1(BookTheaterSeat b, int seats) {
        this.b = b;
        this.seats = seats;
    }

    @Override
    public void run() {
        b.bookSeats(seats);
    }

}

class Thread2 extends Thread {

    BookTheaterSeat b;
    int seats;

    public Thread2(BookTheaterSeat b, int seats) {
        this.b = b;
        this.seats = seats;
    }

    @Override
    public void run() {
        b.bookSeats(seats);
    }

}

public class MovieBookApp {

    public static void main(String[] args) {

        BookTheaterSeat b = new BookTheaterSeat();

        Thread1 thread1 = new Thread1(b, 7);
        thread1.start();
        Thread1 thread2 = new Thread1(b, 6);
        thread2.start();

        BookTheaterSeat b1 = new BookTheaterSeat();

        Thread1 thread3 = new Thread1(b1, 5);
        thread3.start();
        Thread1 thread4 = new Thread1(b1, 7);
        thread4.start();

    }

}

```

}

Explanation

1. **totalSeats is static** → Means all booking counters share the same seat data.
2. **Static synchronized method (bookSeats)** → Ensures **only one thread at a time** can update seats across all booking counters.
3. **Multiple objects (counter1, counter2, counter3)** → Represent multiple railway counters, but all share the same seat count.

Real-World Analogy

Think of **IRCTC**:

- Multiple people try to book tickets from different locations.
- If **two people try to book the last seat at the same time**, only one should succeed.
- **Static synchronization ensures** that the ticket is given to only one user, avoiding duplicate bookings.